1. ТЕОРЕТИЧНІ ОСНОВИ

Нестрогим математичним визначенням поняття «функція» можна прийняти таке визначення.

Функція – це закон, що визначає однозначну відповідність кожному елементу деякої множини *X* одного елементу деякої множини *Y*.

У C++ функція відповідає наведеному вище математичному визначенню, а саме, – це оформлена спеціальним чином самостійна програмна одиниця, яка проектується для розв'язання конкретної задачі, перетворюючи одне або декілька значень не обов'язково того самого типу в одне значення деякого типу[1].

Крім функцій, що входять у стандартні бібліотеки, які можна підключати до розроблюваної програми, програміст має можливість розробляти і використовувати власні функції. Функція оформлюється один раз, а використовується багаторазово в різних місцях програми, виконуючи одні й ті ж самі дії над різними оброблюваними даними. Функції дозволяють структурувати програму; написання великих програм без використання функцій говорить про поганий стиль програмування. Програма не повинна залежати від особливостей реалізації функцій – функції слід писати так, щоб їх можна було переносити з одних програм в інші й створювати бібліотеки функцій.

Таким чином, використання функцій дає такі дві основних переваги:

1) відокремлюється хід розв'язання від конкретної реалізації функції – немає потреби знати, як реалізована функція, потрібно тільки вміти до неї звернутися;

2) опис функції в програмі дається один раз, а використовувати її можна багаторазово.

Функція визначається у відповідності з наступним форматом:

Заголовок функції
Тіло функції

Розглянемо деякі особливості опису функції і звертання до неї.

1. Заголовок функції має такий формат:

тип ім'я_функції (специфікація_формальних_параметрів)

Тут тип – це тип значення, яке є результатом роботи функції (**`char`**, **`int`**, **`double`** і т. д.). Тип може бути не вказаним; у такому разі функція має тип **`int`**.

[1] Функція може не отримувати оброблювані значення із зовнішнього по відношенню до неї середовища, як і не повертати результат своєї роботи.

Якщо функція не повертає ніяке значення, то її тип задається службовим словом **void**;

ім'я_функції – довільно вибраний ідентифікатор, за допомогою якого здійснюється звертання до функції;

специфікація_формальних_параметрів – це список описів формальних параметрів, кожен з яких у найпростішому випадку має такий вигляд:

позначення_типу ім'я_параметру

Окремі специфікації у списку розділяються комою. Список формальних параметрів може закінчуватися комою з наступними трьома крапками (**. . .**), які вказують на те, що при звертанні до функції можна вказати більше фактичних параметрів, ніж їх зазначено в специфікації_формальних_параметрів.

Наприклад:

```
double fun(double a, double b, int d)
```

Параметри можуть мати значення за умовчанням. Значення за умовчанням задається після імені параметру і символу = ; наприклад:

```
double fun(double a, double b = 1.2, int d = 0)
```

2. Тіло функції – це послідовність операторів, укладена у фігурні дужки, тобто це блок (тіло функції може містити описи і визначення). Всередині тіла функції не можна визначити іншу функцію (функції не можуть бути вкладеними одна в одну). Наприклад:

```
{
      double r;
       //. . .
      r = d * (a + b);
      //. . .
}
```

3. Змінні, типи, константи тощо, оголошені в тілі функції, називаються ***локальними***. Ці змінні поза функцією не існують, оскільки під них виділяється пам'ять щоразу при виклику функції, а при виході з функції ці змінні знищуються[1].

4. Формальні параметри є локальними змінними.

---

[1] Якщо локальна змінна описана з модифікатором **static**, то пам'ять під неї відводиться один раз. Значення такої змінної зберігається після виходу з функції і може бути використане при повторному звертанні до функції.

5. Імена локальних змінних, за винятком тих, що описані або визначені у вкладених блоках тіла функції, не можуть збігатися з іменами формальних параметрів (це є наслідком попереднього).

6. Якщо функція не має формальних параметрів, то специфікація_формальних_параметрів або відсутня, або задається службовим словом **`void`**. Дужки в заголовку вказуються в обов'язковому порядку.

7. Якщо функція повертає результат своєї роботи в точку виклику, то це виконується за допомогою оператора

`return` вираз`;`

Цей оператор здійснює дії, які зв'язані з завершенням роботи функції. При цьому значення наведеного в ньому виразу (або константи чи змінної) перетворюється відповідно до типу, указаного в заголовку функції (якщо це можливо).

8. Всі змінні, описані у файлі, що містить текст програми, до визначення функції, також відомі усередині функції. Ці змінні називаються ***глобальними***. У зв'язку із загальнодоступністю глобальних змінних у складних випадках можлива ситуація, коли одна функція може змінити значення глобальної змінної «непомітно» для іншої, результатом чого стає поява помилок, які досить важко виявляються. Щоб уникнути цього, рекомендується обмежувати використання глобальних змінних. Глобальні змінні необхідні в тому випадку, коли програмісту потрібно зробити дані доступними для декількох функцій, а передача даних з однієї функції в іншу є проблематичною.

9. Імена формальних параметрів і локальних змінних можуть збігатися з різними іменами, оголошеними поза функцією. У цьому випадку в тілі функції активними будуть імена, оголошені в ній.

10. Для звертання до функції використовується вираз з операцією «круглі дужки»:

позначення_функції`(`список_фактичних_параметрів`)`

Тут позначення_функції – це або її ім'я, або розіменований вказівник на неї, список_фактичних_параметрів – список змінних, констант, виразів, що розділяються комами. Відповідність між фактичними і формальними параметрами встановлюється по порядку їх взаємного розміщення в списках. При виклику функції дужки вказуються завжди, навіть якщо функція не має параметрів. Приклад звертання:

```
double z = fun(a, 1.5, 2 * n) - 0.2;
```

11. При звертанні до функції під формальні параметри відводиться пам'ять, після чого значення фактичних параметрів переписуються у відповідні

формальні параметри з автоматичним перетворенням типу кожного з них до типу відповідного формального параметра (якщо це можливе). Далі всередені тіла функції здійснюється обробка формальних параметрів. Тому через формальні параметри передача значень з тіла функції в точку виклику неможлива, тим більш, що по закінченні роботи функції формальні параметри знищуються.

12. До першого звертання до функції повинне бути здійснене її визначення або попередній опис за допомогою прототипу, який співпадає з заголовком функції, але при цьому завершується точкою з комою. У прототипі можна не вказувати імена формальних параметрів, обмежившись тільки їхніми типами.

2. ПРИКЛАДИ ПРОГРАМ

Приклад 1. Дано дійсні числа *v* і *w*. Обчислити значення виразу

$$\mathrm{sh}\,(v+\cos(w+1.5))+\mathrm{ch}\,(v+\sin(w+1.5))-e^{\mathrm{sh}\,2+\mathrm{ch}\,\pi},$$

де $\mathrm{sh}\,z$, $\mathrm{ch}\,z$ – відповідно гіперболічний синус і гіперболічний косинус числа *z*, які визначаються такими формулами:

$$\mathrm{sh}\,z=\frac{e^{z}-e^{-z}}{2},\quad \mathrm{ch}\,z=\frac{e^{z}+e^{-z}}{2}.$$

Визначити функції обчислення гіперболічного синуса і гіперболічного косинуса.

Розв'язок.

```
#define _USE_MATH_DEFINES
#include <iostream>
#include <conio.h>
using namespace std;
double Sh(double z);            // Опис (прототип) функції Sh()
double Ch(double z);            // Опис (прототип) функції Ch()
                                 // Визначення головної функції
int main()
{
    double v, w;

    cout << "v = ";
    cin >> v;
    cout << "w = ";
    cin >> w;
    cout << "Value of expression is equal "
```

```
          << Sh(v + cos(w + 1.5)) + Ch(v + sin(w + 1.5))
             - exp(Sh(2) + Ch(M_PI)) << endl;
    cout << "Press any key";
    _getch();
    return 0;
}

double Sh(double z)                    // Визначення функції Sh()
{
    return (exp(z) - exp(-z)) / 2;
}

double Ch(double z)                    // Визначення функції Ch()
{
    return (exp(z) + exp(-z)) / 2;
}
```

Приклад 2. Дано n об'єктів, $n \le 12$. Скільки можна сформувати з них різних сполучень по m ($m \le n$) об'єктів. Число сполучень з n по m визначається за формулою

$$C_n^m = \frac{n!}{(n-m)! \cdot m!}.$$

Розв'язок.

```
#include <iostream>
using namespace std;

int factorial(int);      // Прототип (опис) функції factorial()
                                  // Визначення головної функції
int main()
{
    int m, n;
    int combination;
    cout << "n = ";
    cin >> n;
    cout << "m = ";
    cin >> m;
    combination = factorial(n) / factorial(m) /
                                  factorial(n - m);
    cout << combination << '\n';
    system("pause");
    return 0;
```

```
}
int factorial(int k)          // Визначення функції factorial()
{
    int Result = 1;
    for (int i = 1; i <= k; i++)
        Result *= i;
    return Result;
}
```

Приклад 3. Дано цілочисловий масив, що містить не більше 20 елементів. Чи правда, що вміст масиву однаково читається зліва направо і справа наліво? Для розв'язання задачі використати власну функцію перевірки одно вимірного масиву на його симетричність.

Розв'язок.

```
#include <iostream>
using namespace std;
bool test(int a[], int n);           // Прототип функції test()
                                  // Визначення головної функції
int main()
{
    int n;
    int a[20];
    cout << "n = ";
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cout << "a[" << i << "] = ";
        cin >> a[i];
    }
    if (test(a, n))
        cout << "Yes\n";
    else
        cout << "No\n";
    system("pause");
    return 0;
}

bool test(int a[], int n)          // Визначення функції test()
{
    for (int i = 0; i < n; i++)
        if (a[i] != a[n - 1 - i])
```

```
            return false;
    return true;
}
```

Приклад 4. Обчислити слід (суму елементів, розташованих на головній діагоналі) квадратної матриці розміру не більш ніж 20×20. Визначити функцію обчислення сліду квадратної матриці.

Розв'язок.

```
#include <iostream>
using namespace std;

double trace(double a[20][20], int n);      // Прототип функції
                                    // Визначення головної функції
int main()
{
    int n;
    double a[20][20];
    cout << "n = ";
    cin >> n;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
        {
            cout << "a[" << i << "][" << j << "] = ";
            cin >> a[i][j];
        }
    cout << "\nMatrix\n";
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            cout << a[i][j] << '\t';
        cout << '\n';
    }
    cout << "\nTrace == " << trace(a, n) << '\n';
    system("pause");
    return 0;
}

double trace(double a[20][20], int n)    // Визначення функції
{
    double tr = 0;
    for (int i =0; i < n; i++)
        tr += a[i][i];
```

```
    return tr;
}
```

Приклад 5. Дано два одновимірних масиви *a* і *b*, кожен з яких містить не більш 20 дійсних. Чи правда, що мінімальний елемент масиву *a* більший за мінімальний елемент масиву *b*? Розробити і використати функцію визначення мінімального елементу масиву, яка базується на використанні функції визначення мінімального з двох дійсних чисел.

Розв'язок.

```
#include <iostream>
using namespace std;
                                                // Прототипи функцій
int CompareValues(double a, double b);
int NumberMinArray(double *a, int n);
double MinAB(double a, double b);
                                          // Визначення головної функції
int main()
{
    int m, n;
    double a[20], b[20];
    cout << "Array a\n";
    cout << "m = ";
    cin >> m;
    for (int i = 0; i < m; i++)
    {
        cout << "a[" << i << "] = ";
        cin >> a[i];
    }
    cout << "Array b\n";
    cout << "n = ";
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cout << "b[" << i << "] = ";
        cin >> b[i];
    }
    if (CompareValues(a[NumberMinArray(a, m)],
                      b[NumberMinArray(b, n)]) == 1)
        cout << "Yes\n";
    else
        cout << "No\n";
    system("pause");
```

```
    return 0;
}
                                                    // Визначення функцій
int CompareValues(double a, double b)
{
    if (a > b)
        return 1;
    else
        if (a <b)
            return 2;
    return 0;
}

int NumberMinArray(double *a, int n)
{
    int num = 0;
    for (int i = 1; i < n; i++)
        if (MinAB(a[i], a[num]))
            num = i;
    return num;
}

double MinAB(double a, double b)
{
    if (a < b)
        return a;
    return b;
}
```

3. ВАРІАНТИ ЗАДАЧ

1. Дано натуральне число n. Одержати всі піфагорові трійки натуральних чисел, кожне з яких не перевищує n, тобто всі такі трійки натуральних чисел a, b, c, що $a^2 + b^2 = c^2$ $(a \le b \le c \le n)$. Оформити функцію перевірки того, що дана трійка натуральних чисел є піфагоровою.
2. Дано натуральне число n, дійсні числа a_0, a_1, …, a_n, x. Обчислити $a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$. Визначити функцію піднесення дійсного значення до цілого невід'ємного степеня.
3. Для дійсних значень x з інтервалу $-1 < x < 1$ справедливе співвідношення

$$\operatorname{arctg} x = x - \frac{x^3}{3} + \frac{x^5}{5} - \ldots$$

Визначити функцію з двома параметрами для обчислення значення $\operatorname{arctg} x$ за наведеною вище формулою з урахуванням усіх доданків, що за модулем більші від заданого додатного значення ε.

4. Створити власні функції, призначені для обчислення наближених значень функцій e^x, $\sin x$, $\cos x$ за допомогою розкладання в нескінченні ряди з ураховуванням у сумах членів рядів всіх доданків, що за модулем більші від заданого додатного значення ε:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots,$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \ldots,$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \ldots$$

Протестувати функції при розв'язанні такої задачі. Дано дійсні числа x, μ ($\mu > 0$). Послідовно розглядаючи як ε числа з послідовності 0.5, 0.25, 0.125, …, для кожної з трьох визначених у програмі функцій знайти таке число ε, для якого значення цієї функції вперше буде відхилятися від значення, обчисленого за допомогою відповідної функції з бібліотеки стандартних програм, на величину, що не перевищує μ.

5. Дано ціле число n. Знайти найменше *число Сміта*, що перевищує n. Визначити функцію перевірки цілого числа на те, що воно є числом Сміта, а також функцію перевірки цілого числа на те, що воно є простим.

Примітка. Натуральне число називається числом Сміта, якщо воно не є простим і при цьому сума його цифр дорівнює сумі цифр розкладання цього числа на прості множники. Число 4 937 775 розкладається на прості множники в такий спосіб: 4 937 775 = = 3 × 5 × 5 × 65 837. Сума цифр цього розкладання дорівнює 3 + 5 + 5 + 6 + 5 + 8 + 3 + 7 = 42. Цьому ж числу дорівнює сума цифр самого числа 4 937 775: 4 + 9 + 3 + 7 + 7 + 7 + 5 = 42.

6. Дано дійсні числа *a*, *b*, *c*, *d*. Знайти площу шестикутника, зображеного на рис. 4.1. Визначити функцію обчислення площі трикутника за трьома його сторонами, заданими дійсними числами. Якщо ці числа не дозволяють побудувати трикутник, то результатом роботи функції має бути значення –1.

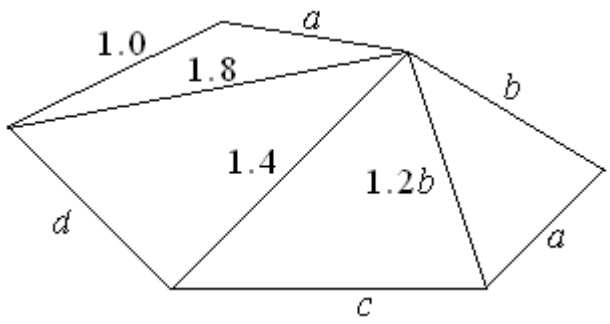


Рис. 4.1

7. Два натуральних числа називаються *дружніми*, якщо кожне з них дорівнює сумі всіх дільників іншого, крім самого цього числа. Дано натуральні числа m, n ($m < n$). Знайти всі пари дружніх чисел у діапазоні від m до n. Визначити функцію перевірки того, що два натуральних числа є дружніми.
8. Дано натуральне число n, цілі числа a_1, a_2, …, a_n. Знайти всі відрізки послідовності a_1, a_2, …, a_n з членів, які йдуть підряд і складаються з довершених чисел. Визначити функцію, що дозволяє розпізнавати довершені числа. **Примітка.** *Довершеним* називають таке натуральне число, що дорівнює сумі всіх своїх дільників, менших від самого числа ($6 = 1 + 2 + 3$).
9. Дано натуральне число n. Для $k = 1, 2, \ldots, n$ одержати суму тих чисел виду $k^3 + (3k + 1) \cdot n$, які є потроєними непарними. Визначити функцію перевірки того, що натуральне число є потроєним непарним.
10. Дано натуральне число n і цілі числа a_1, a_2, …, a_n. Для послідовності a_1, a_2, …, a_n розглянути підпослідовності членів, що йдуть підряд і складаються:
    а) з повних квадратів;
    б) із степенів числа 5;
    в) з простих чисел.

    У кожному випадку одержати найбільшу з довжин розглянутих відрізків. Визначити функції, що дозволяють розпізнавати повні квадрати, степені числа 5, прості числа.
11. Дано два цілочислових одновимірних масиви однакового розміру. Знайти позицію першого елементу, яким відрізняються ці масиви. Описати функцію визначення номера першого елементу, яким відрізняються будь-які два цілочислових одновимірних масиви з однаковою розмірністю. Якщо масиви еквівалентні, то результатом роботи функцій має бути значення –1.
12. Розробити функцію, що дозволяє визначити порядковий номер крайнього праворуч входження заданого цілого числа c в одновимірний цілочисловий масив, що є її параметром. Якщо масив не містить число c, результатом роботи функції має бути значення –1.

13. Розробити функцію, що дозволяє визначити порядковий номер першого входження в заданий одновимірний цілочисловий масив елементів другого одновимірного цілочислового масиву (масиви не обов'язково повинні мати один і той самий розмір). Результатом роботи функції має бути значення –1, якщо перший масив не містить жодного числа, що належить і другому заданому масиву.
14. Описати функцію, що обчислює суму елементів *k*-го стовпця дійсної матриці $\mathbf{A} = (a_{ij})_{mn}$ розміром $m \times n$. Для даної дійсної матриці $\mathbf{A}$ розміром $m \times n$, використовуючи функцію підсумовування елементів стовпця матриці, знайти номер стовпця з мінімальною сумою елементів.
15. Дано натуральне число *n*. З'ясувати, чи є серед чисел n, $n+1$, …, $2n$ близнюки, тобто прості числа, різниця між якими дорівнює 2. Визначити функцію, що дозволяє розпізнавати прості числа.
16. Натуральне число з *n* цифр є *числом Армстронга*, якщо сума його цифр, зведених до *n*-го степеня, дорівнює самому числу (наприклад, $153 = 1^3 + 5^3 + 3^3$). Оформити функцію перевірки того, що натуральне число є числом Армстронга, і скористатися нею для одержання всіх дво-, три-, чотири- і п'ятицифрових чисел Армстронга.
17. Дано парне число $n > 2$. Перевірити для цього числа *гіпотезу Гольдбаха*, яка полягає в тому, що кожне парне *n*, більше 2, подається у вигляді суми двох простих чисел. Визначити функцію, що дозволяє розпізнавати прості числа.
18. Будемо говорити, що натуральне число *v* «краще» за інше натуральне число *w*, якщо сума цифр *v* більша від суми цифр *w*, а в разі рівності сум їхніх цифр число *v* «кращім» буде тоді, коли $v < w$. Наприклад, число 124 «краще» від числа 123, тому що в першого з них сума цифр дорівнює семи, а в другого – шести. Водночас число 3 «краще» за 111, тому що в них рівні суми цифр, але $3 < 111$.

    Дано натуральне число *n*. Знайти такий дільник цього числа, що є «кращим» за будь-який інший дільник. Визначити функцію знаходження «найкращого» дільника натурального числа, заданого як її параметр.
19. Дано натуральне число *n* і масив *a* з *n* натуральних чисел. Одержати масив *b* з «найкращих» дільників масиву *a* (натуральне число *v* будемо вважати «кращім» за інше натуральне число *w*, якщо сума цифр *v* більша від суми цифр *w*, а в разі рівності сум їхніх цифр число *v* кращім буде тоді, коли $v < w$). Визначити три функції – функцію обчислення суми цифр цілого числа, функцію перевірки того, що одне натуральне число «краще» від іншого, і функцію відшукання «найкращого» дільника натурального числа.